You are given a task to integrate an existing React component in the codebase

The codebase should support:
- shadcn project structure
- Tailwind CSS
- Typescript

If it doesn't, provide instructions on how to setup project via shadcn CLI, install Tailwind or Typescript.

Determine the default path for components and styles.
If default path for components is not /components/ui, provide instructions on why it's important to create this folder
Copy-paste this component to /components/ui folder:
```tsx
animated-glassy-pricing.tsx
import React, { useRef, useEffect, useState } from 'react';
import { RippleButton } from "@/components/ui/multi-type-ripple-buttons";
// --- Internal Helper Components (Not exported) --- //

const CheckIcon = ({ className }: { className?: string }) => (
  <svg
    xmlns="http://www.w3.org/2000/svg"
    width="16" height="16" viewBox="0 0 24 24"
    fill="none" stroke="currentColor" strokeWidth="3"
    strokeLinecap="round" strokeLinejoin="round"
    className={className}
  >
    <path d="M20 6 9 17l-5-5" />
  </svg>
);

const ShaderCanvas = () => {
  const canvasRef = useRef<HTMLCanvasElement | null>(null);
  const glProgramRef = useRef<WebGLProgram | null>(null);
  const glBgColorLocationRef = useRef<WebGLUniformLocation | null>(null);
  const glRef = useRef<WebGLRenderingContext | null>(null);
  const [backgroundColor, setBackgroundColor] = useState([1.0, 1.0, 1.0]);

  useEffect(() => {
```

```
    const root = document.documentElement;
    const updateColor = () => {
      const isDark = root.classList.contains('dark');
      setBackgroundColor(isDark ? [0, 0, 0] : [1.0, 1.0, 1.0]);
    };
    updateColor();
    const observer = new MutationObserver((mutationsList) => {
      for (const mutation of mutationsList) {
        if (mutation.type === 'attributes' && mutation.attributeName === 'class') {
          updateColor();
        }
      }
    });
    observer.observe(root, { attributes: true });
    return () => observer.disconnect();
  }, []);

  useEffect(() => {
    const gl = glRef.current;
    const program = glProgramRef.current;
    const location = glBgColorLocationRef.current;
    if (gl && program && location) {
      gl.useProgram(program);
      gl.uniform3fv(location, new Float32Array(backgroundColor));
    }
  }, [backgroundColor]);

  useEffect(() => {
    const canvas = canvasRef.current;
    if (!canvas) return;
    const gl = canvas.getContext('webgl');
    if (!gl) { console.error("WebGL not supported"); return; }
    glRef.current = gl;

    const vertexShaderSource = `attribute vec2 aPosition; void main() { gl_Position =
vec4(aPosition, 0.0, 1.0); }`;
    const fragmentShaderSource = `
      precision highp float;
      uniform float iTime;
      uniform vec2 iResolution;
```

```
    uniform vec3 uBackgroundColor;
    mat2 rotate2d(float angle){ float c=cos(angle),s=sin(angle); return mat2(c,-s,s,c); }
    float variation(vec2 v1,vec2 v2,float strength,float speed){ return
sin(dot(normalize(v1),normalize(v2))*strength+iTime*speed)/100.0; }
    vec3 paintCircle(vec2 uv,vec2 center,float rad,float width){
      vec2 diff = center-uv;
      float len = length(diff);
      len += variation(diff,vec2(0.,1.),5.,2.);
      len -= variation(diff,vec2(1.,0.),5.,2.);
      float circle = smoothstep(rad-width,rad,len)-smoothstep(rad,rad+width,len);
      return vec3(circle);
    }
    void main(){
      vec2 uv = gl_FragCoord.xy/iResolution.xy;
      uv.x *= 1.5; uv.x -= 0.25;
      float mask = 0.0;
      float radius = .35;
      vec2 center = vec2(.5);
      mask += paintCircle(uv,center,radius,.035).r;
      mask += paintCircle(uv,center,radius-.018,.01).r;
      mask += paintCircle(uv,center,radius+.018,.005).r;
      vec2 v=rotate2d(iTime)*uv;
      vec3 foregroundColor=vec3(v.x,v.y,.7-v.y*v.x);
      vec3 color=mix(uBackgroundColor,foregroundColor,mask);
      color=mix(color,vec3(1.),paintCircle(uv,center,radius,.003).r);
      gl_FragColor=vec4(color,1.);
    }`;

  const compileShader = (type: number, source: string) => {
    const shader = gl.createShader(type);
    if (!shader) throw new Error("Could not create shader");
    gl.shaderSource(shader, source);
    gl.compileShader(shader);
    if (!gl.getShaderParameter(shader, gl.COMPILE_STATUS)) {
      throw new Error(gl.getShaderInfoLog(shader) || "Shader compilation error");
    }
    return shader;
  };

  const program = gl.createProgram();
```

```
  if (!program) throw new Error("Could not create program");
  const vertexShader = compileShader(gl.VERTEX_SHADER, vertexShaderSource);
  const fragmentShader = compileShader(gl.FRAGMENT_SHADER,
fragmentShaderSource);
  gl.attachShader(program, vertexShader);
  gl.attachShader(program, fragmentShader);
  gl.linkProgram(program);
  gl.useProgram(program);
  glProgramRef.current = program;

  const buffer = gl.createBuffer();
  gl.bindBuffer(gl.ARRAY_BUFFER, buffer);
  gl.bufferData(gl.ARRAY_BUFFER, new Float32Array([-1, -1, 1, -1, -1, 1, -1, 1, 1, -1, 1,
1]), gl.STATIC_DRAW);
  const aPosition = gl.getAttribLocation(program, 'aPosition');
  gl.enableVertexAttribArray(aPosition);
  gl.vertexAttribPointer(aPosition, 2, gl.FLOAT, false, 0, 0);

  const iTimeLoc = gl.getUniformLocation(program, 'iTime');
  const iResLoc = gl.getUniformLocation(program, 'iResolution');
  glBgColorLocationRef.current = gl.getUniformLocation(program, 'uBackgroundColor');
  gl.uniform3fv(glBgColorLocationRef.current, new Float32Array(backgroundColor));

  let animationFrameId: number;
  const render = (time: number) => {
   gl.uniform1f(iTimeLoc, time * 0.001);
   gl.uniform2f(iResLoc, canvas.width, canvas.height);
   gl.drawArrays(gl.TRIANGLES, 0, 6);
   animationFrameId = requestAnimationFrame(render);
  };
  const handleResize = () => {
   canvas.width = window.innerWidth;
   canvas.height = window.innerHeight;
   gl.viewport(0, 0, gl.drawingBufferWidth, gl.drawingBufferHeight);
  };
  handleResize();
  window.addEventListener('resize', handleResize);
  animationFrameId = requestAnimationFrame(render);
  return () => {
   window.removeEventListener('resize', handleResize);
```

```
    cancelAnimationFrame(animationFrameId);
   };
  }, []);

  return <canvas ref={canvasRef} className="fixed top-0 left-0 w-full h-full block z-0 bg-
background" />;
};


// --- EXPORTED Building Blocks --- //

/**
 * We export the Props interface so you can easily type the data for your plans.
 */
export interface PricingCardProps {
  planName: string;
  description: string;
  price: string;
  features: string[];
  buttonText: string;
  isPopular?: boolean;
  buttonVariant?: 'primary' | 'secondary';
}

/**
 * We export the PricingCard component itself in case you want to use it elsewhere.
 */
export const PricingCard = ({
  planName, description, price, features, buttonText, isPopular = false, buttonVariant =
'primary'
}: PricingCardProps) => {
  const cardClasses = `
   backdrop-blur-[14px] bg-gradient-to-br rounded-2xl shadow-xl flex-1 max-w-xs px-7
py-8 flex flex-col transition-all duration-300
   from-black/5 to-black/0 border border-black/10
   dark:from-white/10 dark:to-white/5 dark:border-white/10 dark:backdrop-brightness-
[0.91]
   ${isPopular ? 'scale-105 relative ring-2 ring-cyan-400/20 dark:from-white/20 dark:to-
white/10 dark:border-cyan-400/30 shadow-2xl' : ''}
  `;
```

```
  const buttonClasses = `
    mt-auto w-full py-2.5 rounded-xl font-semibold text-[14px] transition font-sans
    ${buttonVariant === 'primary'
      ? 'bg-cyan-400 hover:bg-cyan-300 text-foreground'
      : 'bg-black/10 hover:bg-black/20 text-foreground border border-black/20 dark:bg-
white/10 dark:hover:bg-white/20 dark:text-white dark:border-white/20'
    }
  `;

  return (
    <div className={cardClasses.trim()}>
      {isPopular && (
        <div className="absolute -top-4 right-4 px-3 py-1 text-[12px] font-semibold
rounded-full bg-cyan-400 text-foreground dark:text-black">
          Most Popular
        </div>
      )}
      <div className="mb-3">
        <h2 className="text-[48px] font-extralight tracking-[-0.03em] text-foreground
font-display">{planName}</h2>
        <p className="text-[16px] text-foreground/70 mt-1 font-sans">{description}</p>
      </div>
      <div className="my-6 flex items-baseline gap-2">
        <span className="text-[48px] font-extralight text-foreground font-
display">${price}</span>
        <span className="text-[14px] text-foreground/70 font-sans">/mo</span>
      </div>
      <div className="card-divider w-full mb-5 h-px bg-[linear-
gradient(90deg,transparent,rgba(0,0,0,0.1)_50%,transparent)] dark:bg-[linear-
gradient(90deg,transparent,rgba(255,255,255,0.09)_20%,rgba(255,255,255,0.22)_50%,r
gba(255,255,255,0.09)_80%,transparent)]"></div>
      <ul className="flex flex-col gap-2 text-[14px] text-foreground/90 mb-6 font-sans">
        {features.map((feature, index) => (
          <li key={index} className="flex items-center gap-2">
            <CheckIcon className="text-cyan-400 w-4 h-4" /> {feature}
          </li>
        ))}
      </ul>
      <RippleButton className={buttonClasses.trim()}>{buttonText}</RippleButton>
    </div>
```

```
  );
};


// --- EXPORTED Customizable Page Component --- //

interface ModernPricingPageProps {
  /** The main title. Can be a string or a ReactNode for more complex content. */
  title: React.ReactNode;
  /** The subtitle text appearing below the main title. */
  subtitle: React.ReactNode;
  /** An array of plan objects that conform to PricingCardProps. */
  plans: PricingCardProps[];
  /** Whether to show the animated WebGL background. Defaults to true. */
  showAnimatedBackground?: boolean;
}

export const ModernPricingPage = ({
  title,
  subtitle,
  plans,
  showAnimatedBackground = true,
}: ModernPricingPageProps) => {
  return (
    <div className="bg-background text-foreground min-h-screen w-full overflow-x-
hidden">
      {showAnimatedBackground && <ShaderCanvas />}
      <main className="relative w-full min-h-screen flex flex-col items-center justify-
center px-4 py-8">
        <div className="w-full max-w-5xl mx-auto text-center mb-14">
          <h1 className="text-[48px] md:text-[64px] font-extralight leading-tight tracking-[-
0.03em] bg-clip-text text-transparent bg-gradient-to-r from-slate-900 via-cyan-500 to-
blue-600 dark:from-white dark:via-cyan-300 dark:to-blue-400 font-display">
            {title}
          </h1>
          <p className="mt-3 text-[16px] md:text-[20px] text-foreground/80 max-w-2xl
mx-auto font-sans">
            {subtitle}
          </p>
        </div>
```

```
      <div className="flex flex-col md:flex-row gap-8 md:gap-6 justify-center items-center w-full max-w-4xl">
        {plans.map((plan) => <PricingCard key={plan.planName} {...plan} />)}
      </div>
    </main>
  </div>
 );
};
```

demo.tsx

```
import { ModernPricingPage, PricingCardProps  } from "@/components/ui/animated-glassy-pricing";

const myPricingPlans: PricingCardProps[] = [
 {
  planName: 'Basic',
  description: 'Perfect for personal projects and hobbyists.',
  price: '0',
  features: ['1 User', '1GB Storage', 'Community Forum'],
  buttonText: 'Get Started',
  buttonVariant: 'secondary'
 },
 {
  planName: 'Team',
  description: 'Collaborate with your team on multiple projects.',
  price: '49',
  features: ['10 Users', '100GB Storage', 'Email Support', 'Shared Workspaces'],
  buttonText: 'Choose Team Plan',
  isPopular: true,
  buttonVariant: 'primary'
 },
 {
  planName: 'Agency',
  description: 'Manage all your clients under one roof.',
  price: '149',
  features: ['Unlimited Users', '1TB Storage', 'Dedicated Support', 'Client Invoicing'],
  buttonText: 'Contact Us',
  buttonVariant: 'primary'
 },
];
```

```
const Default = () => {
  return <ModernPricingPage
      title={
        <>
          Find the <span className="text-cyan-400">Perfect Plan</span> for Your
Business
        </>
      }
      subtitle="Start for free, then grow with us. Flexible plans for projects of all sizes."
      plans={myPricingPlans}
      showAnimatedBackground={true}
    />
;
};

export { Default };
```

Copy-paste these files for dependencies:
```tsx
easemize/multi-type-ripple-buttons
import React, { ReactNode, useState, useMemo, MouseEvent, CSSProperties } from
'react';

interface RippleState {
  key: number;
  x: number;
  y: number;
  size: number;
  color: string;
}

interface RippleButtonProps {
  children: ReactNode;
  onClick?: (event: MouseEvent<HTMLButtonElement>) => void;
  className?: string;
  disabled?: boolean;
  variant?: 'default' | 'hover' | 'ghost' | 'hoverborder';
  rippleColor?: string; // User override for the JS click ripple color
```

```
  rippleDuration?: number; // Duration for the JS click ripple (all variants)

  // For 'hover' variant
  hoverBaseColor?: string;
  hoverRippleColor?: string;

  // For 'hoverborder' variant
  hoverBorderEffectColor?: string; // Color of the visual effect forming the border
  hoverBorderEffectThickness?: string; // Thickness of the border effect (e.g., "0.3em",
"2px")
}

const hexToRgba = (hex: string, alpha: number): string => {
  let hexValue = hex.startsWith('#') ? hex.slice(1) : hex;
  if (hexValue.length === 3) {
    hexValue = hexValue.split('').map(char => char + char).join('');
  }
  const r = parseInt(hexValue.slice(0, 2), 16);
  const g = parseInt(hexValue.slice(2, 4), 16);
  const b = parseInt(hexValue.slice(4, 6), 16);
  return `rgba(${r}, ${g}, ${b}, ${alpha})`;
};

const GRID_HOVER_NUM_COLS = 36;
const GRID_HOVER_NUM_ROWS = 12;
const GRID_HOVER_TOTAL_CELLS = GRID_HOVER_NUM_COLS *
GRID_HOVER_NUM_ROWS;
const GRID_HOVER_RIPPLE_EFFECT_SIZE = "18.973665961em";

const JS_RIPPLE_KEYFRAMES = `
  @keyframes js-ripple-animation {
    0% { transform: scale(0); opacity: 1; }
    100% { transform: scale(1); opacity: 0; }
  }
  .animate-js-ripple-effect {
    animation: js-ripple-animation var(--ripple-duration) ease-out forwards;
  }
`;

const RippleButton: React.FC<RippleButtonProps> = ({
```

```
  children,
  onClick,
  className = '',
  disabled = false,
  variant = 'default',
  rippleColor: userProvidedRippleColor,
  rippleDuration = 600,
  hoverBaseColor = '#6996e2',
  hoverRippleColor: customHoverRippleColor,
  hoverBorderEffectColor = '#6996e277',
  hoverBorderEffectThickness = '0.3em',
}) => {
  const [jsRipples, setJsRipples] = useState<RippleState[]>([]);

  const determinedJsRippleColor = useMemo(() => {
    if (userProvidedRippleColor) {
      return userProvidedRippleColor;
    }
    return 'var(--button-ripple-color, rgba(0, 0, 0, 0.1))';
  }, [userProvidedRippleColor]);

  const dynamicGridHoverStyles = useMemo(() => {
    let nthChildHoverRules = '';
    const cellDim = 0.25;
    const initialTopOffset = 0.125;
    const initialLeftOffset = 0.1875;

    // Standardized hover transition duration for width and height
    const hoverEffectDuration = '0.9s'; // CHANGED: Standardized to 0.9s

    for (let r = 0; r < GRID_HOVER_NUM_ROWS; r++) {
      for (let c = 0; c < GRID_HOVER_NUM_COLS; c++) {
        const childIndex = r * GRID_HOVER_NUM_COLS + c + 1;
        const topPos = initialTopOffset + r * cellDim;
        const leftPos = initialLeftOffset + c * cellDim;

        if (variant === 'hover') {
          nthChildHoverRules += `
            .hover-variant-grid-cell:nth-child(${childIndex}):hover ~ .hover-variant-visual-
ripple {
```

```
          top: ${topPos}em; left: ${leftPos}em;
          transition: width ${hoverEffectDuration} ease, height ${hoverEffectDuration}
ease, top 0s linear, left 0s linear;
        }`;
      } else if (variant === 'hoverborder') {
        nthChildHoverRules += `
          .hoverborder-variant-grid-cell:nth-child(${childIndex}):hover ~ .hoverborder-
variant-visual-ripple {
          top: ${topPos}em; left: ${leftPos}em;
          transition: width ${hoverEffectDuration} ease-out, height ${hoverEffectDuration}
ease-out, top 0s linear, left 0s linear;
        }`; // Using ease-out for hoverborder as it was before, just changed duration
      }
    }
  }

  if (variant === 'hover') {
    const actualHoverRippleColor = customHoverRippleColor
      ? customHoverRippleColor
      : hexToRgba(hoverBaseColor, 0.466);
    return `
      .hover-variant-visual-ripple {
        background-color: ${actualHoverRippleColor};
        transition: width ${hoverEffectDuration} ease, height ${hoverEffectDuration} ease,
top 99999s linear, left 99999s linear;
      }
      .hover-variant-grid-cell:hover ~ .hover-variant-visual-ripple {
        width: ${GRID_HOVER_RIPPLE_EFFECT_SIZE}; height:
${GRID_HOVER_RIPPLE_EFFECT_SIZE};
      }
      ${nthChildHoverRules}
    `;
  } else if (variant === 'hoverborder') {
    return `
      .hoverborder-variant-ripple-container {
        padding: ${hoverBorderEffectThickness};
        mask: linear-gradient(#fff 0 0) content-box, linear-gradient(#fff 0 0);
        mask-composite: exclude;
        -webkit-mask: linear-gradient(#fff 0 0) content-box, linear-gradient(#fff 0 0);
        -webkit-mask-composite: xor;
```

```
      }
      .hoverborder-variant-visual-ripple {
        background-color: ${hoverBorderEffectColor};
        /* Ensure the base transition also uses the standardized duration for width/height
*/
        transition: width ${hoverEffectDuration} ease-out, height ${hoverEffectDuration}
ease-out, top 99999s linear, left 9999s linear;
      }
      .hoverborder-variant-grid-cell:hover ~ .hoverborder-variant-visual-ripple {
        width: ${GRID_HOVER_RIPPLE_EFFECT_SIZE}; height:
${GRID_HOVER_RIPPLE_EFFECT_SIZE};
      }
      ${nthChildHoverRules}
    `;
   }
   return '';
  }, [variant, hoverBaseColor, customHoverRippleColor, hoverBorderEffectColor,
hoverBorderEffectThickness]);

  const createJsRipple = (event: MouseEvent<HTMLButtonElement>) => {
   const button = event.currentTarget;
   const rect = button.getBoundingClientRect();
   const size = Math.max(rect.width, rect.height) * 2;
   const x = event.clientX - rect.left - size / 2;
   const y = event.clientY - rect.top - size / 2;
   const newRipple: RippleState = { key: Date.now(), x, y, size, color:
determinedJsRippleColor };
   setJsRipples(prev => [...prev, newRipple]);
   setTimeout(() => {
    setJsRipples(currentRipples => currentRipples.filter(r => r.key !== newRipple.key));
   }, rippleDuration);
  };

  const handleButtonClick = (event: MouseEvent<HTMLButtonElement>) => {
   if (!disabled) {
    createJsRipple(event);
    if (onClick) onClick(event);
   }
  };
```

```
  const jsRippleElements = (
    <div className="absolute inset-0 pointer-events-none z-[5]">
      {jsRipples.map(ripple => (
        <span
          key={ripple.key}
          className="absolute rounded-full animate-js-ripple-effect"
          style={{
            left: ripple.x, top: ripple.y, width: ripple.size, height: ripple.size,
            backgroundColor: ripple.color,
            ['--ripple-duration' as any]: `${rippleDuration}ms`,
          } as CSSProperties}
        />
      ))}
    </div>
  );

  if (variant === 'hover') {
    const hoverButtonFinalClassName = [
      "relative", "rounded-lg", "text-lg", "px-4", "py-2",
      "border-none", "bg-transparent", "isolate", "overflow-hidden", "cursor-pointer",
      disabled ? "opacity-50 cursor-not-allowed pointer-events-none" : "",
      className,
    ].filter(Boolean).join(" ");
    return (
      <>
        <style dangerouslySetInnerHTML={{ __html: JS_RIPPLE_KEYFRAMES }} />
        <style dangerouslySetInnerHTML={{ __html: dynamicGridHoverStyles }} />
        <button className={hoverButtonFinalClassName} onClick={handleButtonClick}
disabled={disabled}>
          <span className="relative z-[10] pointer-events-none">{children}</span>
          {jsRippleElements}
          <div
            className="hover-variant-grid-container absolute inset-0 grid overflow-hidden
pointer-events-none z-[0]"
            style={{ gridTemplateColumns: `repeat(${GRID_HOVER_NUM_COLS}, 0.25em)` }}
          >
            {Array.from({ length: GRID_HOVER_TOTAL_CELLS }, (_, index) => (
              <span key={index} className="hover-variant-grid-cell relative flex justify-center
items-center pointer-events-auto" />
            ))}
```

```
        <div className="hover-variant-visual-ripple pointer-events-none absolute w-0 h-0 rounded-full transform -translate-x-1/2 -translate-y-1/2 top-0 left-0 z-[-1]" />
        </div>
      </button>
    </>
  );
 }

 if (variant === 'hoverborder') {
  const hoverBorderButtonFinalClassName = [
   "relative", "rounded-lg", "overflow-hidden", "text-lg", "px-4", "py-2",
   "border-none", "bg-transparent", "isolate", "cursor-pointer",
   disabled ? "opacity-50 cursor-not-allowed pointer-events-none" : "",
   className,
  ].filter(Boolean).join(" ");

  return (
   <>
    <style dangerouslySetInnerHTML={{ __html: JS_RIPPLE_KEYFRAMES }} />
    <style dangerouslySetInnerHTML={{ __html: dynamicGridHoverStyles }} />
    <button
     className={hoverBorderButtonFinalClassName}
     onClick={handleButtonClick}
     disabled={disabled}
    >
     <span className="relative z-[10] pointer-events-none">{children}</span>
     {jsRippleElements}
     <div
      className="hoverborder-variant-ripple-container absolute inset-0 grid rounded-[0.8em] overflow-hidden pointer-events-none z-[0]"
      style={{ gridTemplateColumns: `repeat(${GRID_HOVER_NUM_COLS}, 0.25em)` }}
     >
      {Array.from({ length: GRID_HOVER_TOTAL_CELLS }, (_, index) => (
       <span
        key={index}
        className="hoverborder-variant-grid-cell relative flex justify-center items-center pointer-events-auto"
       />
      ))}
```

```
          <div className="hoverborder-variant-visual-ripple pointer-events-none absolute
w-0 h-0 rounded-full transform -translate-x-1/2 -translate-y-1/2 top-0 left-0 z-[-1]" />
        </div>
      </button>
    </>
  );
 }

 if (variant === 'ghost') {
  const ghostButtonFinalClassName = [
    "relative", "border-none", "bg-transparent", "isolate", "overflow-hidden", "cursor-
pointer",
    "px-4", "py-2", "rounded-lg", "text-lg",
    disabled ? "opacity-50 cursor-not-allowed pointer-events-none" : "",
    className,
  ].filter(Boolean).join(" ");
  return (
    <>
      <style dangerouslySetInnerHTML={{ __html: JS_RIPPLE_KEYFRAMES }} />
      <button className={ghostButtonFinalClassName} onClick={handleButtonClick}
disabled={disabled}>
        <span className="relative z-10 pointer-events-none">{children}</span>
        {jsRippleElements}
      </button>
    </>
  );
 }

 // Default variant
 const baseClasses = "relative border-none overflow-hidden isolate transition-all
duration-200 cursor-pointer px-4 py-2 bg-blue-600 hover:opacity-90 text-white
rounded-lg";
 const disabledClasses = disabled ? "opacity-50 cursor-not-allowed" : "";
 const buttonClasses = `${baseClasses} ${disabledClasses} ${className}`;
 return (
  <>
    <style dangerouslySetInnerHTML={{ __html: JS_RIPPLE_KEYFRAMES }} />
    <button className={buttonClasses} onClick={handleButtonClick}
disabled={disabled}>
      <span className="relative z-[1] pointer-events-none">{children}</span>
```

```
      {jsRippleElements}
    </button>
  </>
 );
};

export { RippleButton };
```

Extend existing Tailwind 4 index.css with this code (or if project uses Tailwind 3, extend tailwind.config.js or globals.css):

```css
@import "tailwindcss";
@import "tw-animate-css";

@theme inline {
 --radius-none: 0px;
 --radius-xs: 0.125rem;
 --radius-default: 0.375rem;
 --radius-2xl: 1.5rem;
 --radius-3xl: 1.875rem;
 --radius-full: 9999px;
 --color-button-ripple-color: var(----button-ripple-color);
}

:root {
 --button-ripple-color: oklch(0.145 0 0 / 0.3);
}

.dark {
 --background: oklch(0.10 0 0);
 --button-ripple-color: oklch(0.985 0 0 / 0.5);
}

```

Implementation Guidelines

1. Analyze the component structure and identify all required dependencies
2. Review the component's argumens and state
3. Identify any required context providers or hooks and install them

4. Questions to Ask
- What data/props will be passed to this component?
- Are there any specific state management requirements?
- Are there any required assets (images, icons, etc.)?
- What is the expected responsive behavior?
- What is the best place to use this component in the app?

Steps to integrate
0. Copy paste all the code above in the correct directories
1. Install external dependencies
2. Fill image assets with Unsplash stock images you know exist
3. Use lucide-react icons for svgs or logos if component requires them